

REST API

MagistralaCAN4

Zdalne sterowanie i monitoring przez HTTP

Port 8080 | JSON | QTcpServer

CustomLabs

May 8, 2026

Contents

1	Co to jest REST API?	3
2	Dlaczego REST API jest ważne?	3
2.1	Zdalne sterowanie	3
2.2	Integracja z zewnętrznymi systemami	3
2.3	Lekkość i prostota	3
2.4	Uzupełnienie WebSocket	3
3	Dlaczego warto tego używać?	4
3.1	Scenariusz 1 — Warsztat	4
3.2	Scenariusz 2 — Automatyzacja testów	4
3.3	Scenariusz 3 — Monitoring 24/7	4
4	Jak używać — krok po kroku	4
4.1	Krok 1: Uruchom REST API	4
4.2	Krok 2: Sprawdź status	4
4.3	Krok 3: Pobierz ramki	5
4.4	Krok 4: Steruj sniffingiem	5
4.5	Krok 5: Dashboard HTML	5
5	Architektura techniczna	5
6	Endpointy — pełne zestawienie	6
7	Bezpieczeństwo	6

1 Co to jest REST API?

REST (Representational State Transfer) API to interfejs komunikacyjny oparty na protokole **HTTP**. Działa na zasadzie żądań i odpowiedzi — klient wysyła zapytanie (GET, POST), serwer zwraca dane w formacie **JSON**.

W MagistralaCAN4 REST API nasłuchuje na porcie **8080** i udostępnia cztery endpointy:

GET /api/status	statystyki aplikacji
GET /api/frames	ostatnie ramki CAN
POST /api/start	uruchom sniffing
POST /api/stop	zatrzymaj sniffing

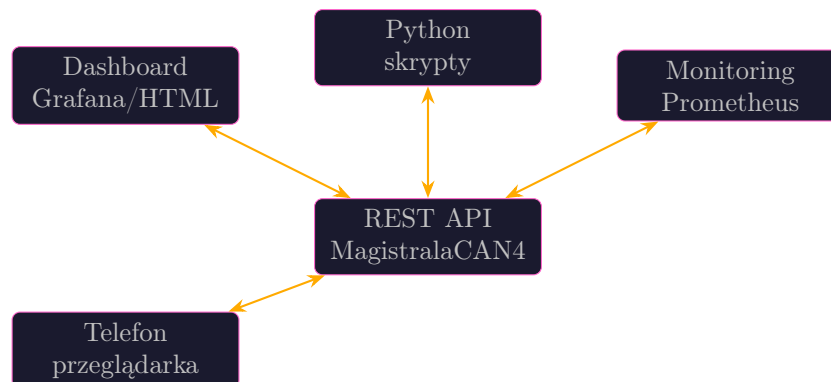
2 Dlaczego REST API jest ważne?

2.1 Zdalne sterowanie

Listing 1: Zdalne uruchomienie sniffingu przez curl

```
curl -X POST http://192.168.1.10:8080/api/start
curl http://192.168.1.10:8080/api/status
```

2.2 Integracja z zewnętrznymi systemami



2.3 Lekkość i prostota

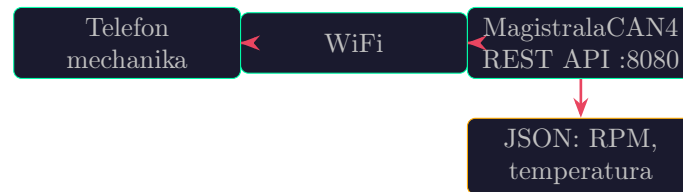
REST API w MagistralaCAN4 to tylko ~100 linii kodu (`HttpRestServer`), żadnych zewnętrznych zależności.

2.4 Uzupełnienie WebSocket

darkbg Cecha	Opis
WebSocket	Strumień ciągły, JSON dla każdej ramki, port 9000 (WSS)
REST API	Żądanie→odpowiedź, JSON agregowany, port 8080 (HTTP)

3 Dlaczego warto tego używać?

3.1 Scenariusz 1 — Warsztat



3.2 Scenariusz 2 — Automatyzacja testów

Listing 2: Automatyzacja testów w Pythonie

```

import requests, time
requests.post("http://localhost:8080/api/start")
time.sleep(10)
data = requests.get("http://localhost:8080/api/frames").json()
requests.post("http://localhost:8080/api/stop")
print(f"Zebrano {len(data)} ramek")
  
```

3.3 Scenariusz 3 — Monitoring 24/7

Co 30 sekund sprawdza `/api/status` i alarmuje gdy FPS spadnie poniżej progu.

4 Jak używać — krok po kroku

4.1 Krok 1: Uruchom REST API

W MagistralaCAN4 kliknij przycisk [REST] REST API w górnym toolbarze. Serwer nasłuchuje na porcie 8080.

4.2 Krok 2: Sprawdź status

Listing 3: curl `/api/status`

```
curl http://localhost:8080/api/status
```

Listing 4: Odpowiedź JSON

```

{
  "running": true,
  "frames": 1247,
  "fps": 82.0,
  "uniqueIds": 12
}
  
```

darkbg Pole	Znaczenie
running	Czy sniffing jest aktywny
frames	Liczba ramek w tabeli
fps	Ramki na sekundę (średnia z 500ms)
uniqueIds	Liczba unikalnych CAN ID

4.3 Krok 3: Pobierz ramki

Listing 5: curl /api/frames

```
curl http://localhost:8080/api/frames
```

4.4 Krok 4: Steruj sniffingiem

Listing 6: Start / Stop

```
curl -X POST http://localhost:8080/api/start
curl -X POST http://localhost:8080/api/stop
```

4.5 Krok 5: Dashboard HTML

Listing 7: Prosty dashboard w HTML/JS

```
<!DOCTYPE html><html><body>
<h1>MagistralaCAN4</h1><div id="status"></div>
<script>
setInterval(async () => {
  const r = await fetch('http://192.168.1.10:8080/api/status');
  const d = await r.json();
  document.getElementById('status').innerHTML =
    'Ramki: ${d.frames} | FPS: ${d.fps}';
}, 1000);
</script></body></html>
```

5 Architektura techniczna

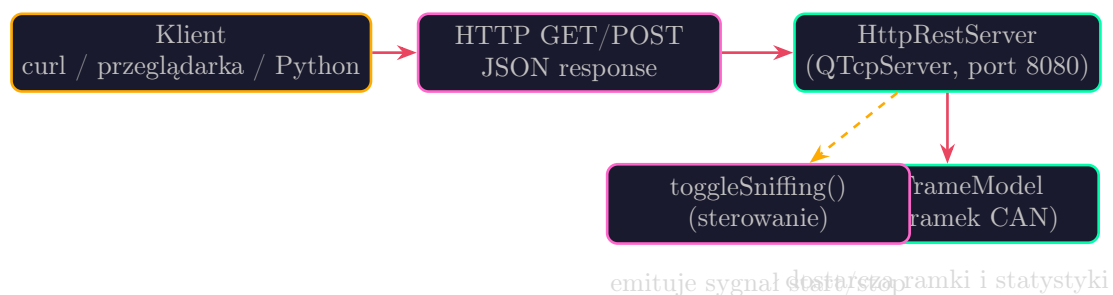
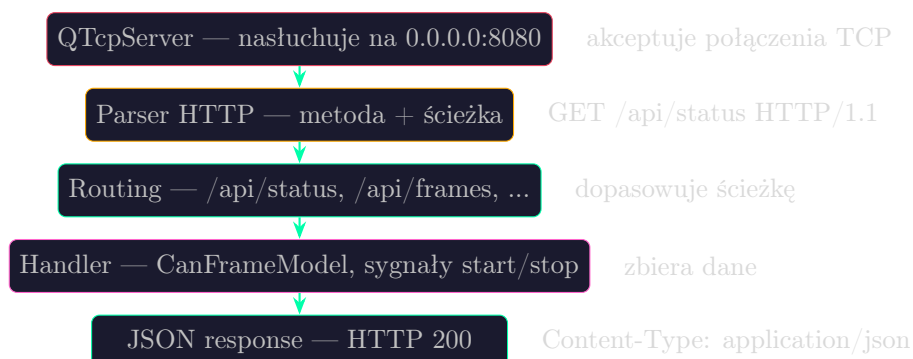


Figure 1: Architektura REST API — przepływ żądania



6 Endpointy — pełne zestawienie

GET /api/status	Statystyki aplikacji JSON: running,frames,fps,uniqueIds	
GET /api/frames	Ostatnie 100 ramek JSON array: id,dlc,timestamp,data	
POST /api/start	Uruchom sniffing	"status":"ok"
POST /api/stop	Zatrzymaj sniffing	"status":"ok"

7 Bezpieczeństwo

- REST API nasłuchuje na 0.0.0.0:8080 — dostępne z sieci
- Brak wbudowanej autoryzacji (założenie: zaufana sieć LAN)
- Ograniczenie przez firewall: `sudo ufw allow from 192.168.1.0/24 to any port 8080`
- Przyszłościowo: token Authorization: Bearer <token>